

In Python, variables are used to store data.

Example:

```
name = "Krish"  
age = 20  
price = 99.5
```

Here, `name`, `age`, and `price` are variable names.

Rules for Declaring Variable Names in Python

1. Variable name must start with a letter or underscore _

✅ Correct:

```
name = "Krish"  
_age = 20
```

❌ Incorrect:

```
1name = "Krish"
```

2. Variable names can contain letters, numbers, and underscores

✅ Correct:

```
student1 = "Raj"  
user_name = "Krish"
```

❌ Incorrect:

```
user-name = "Krish"    # Hyphen not allowed  
user name = "Krish"    # Space not allowed
```

3. Variable names are case-sensitive

```
age = 20  
Age = 25
```

`age` and `Age` are different variables.

4. Python keywords cannot be used as variable names

❌ Incorrect:

```
class = 10
for = 5
if = 7
```

Because `class`, `for`, and `if` are reserved keywords in Python.

5. Special characters are not allowed

✗ Incorrect:

```
price@ = 100
name# = "abc"
```

6. Use meaningful variable names (Best Practice)

✓ Good:

```
student_name = "Krish"
total_marks = 450
```

✗ Bad:

```
a = "Krish"
x = 450
```

Python Naming Convention

Python commonly uses **snake_case** for variable names.

✓ Recommended:

```
student_name = "Krish"
total_marks = 500
```

✗ Not Recommended:

```
studentName = "Krish"
TotalMarks = 500
```

Example Program

```
student_name = "Krish"
student_age = 20
college_name = "ABC College"

print(student_name)
```

```
print(student_age)
print(college_name)
```

Quick Summary

A Python variable name:

- Must start with a letter or _
- Cannot start with a number
- Cannot contain spaces
- Cannot contain special symbols
- Cannot use Python keywords
- Should be meaningful and readable

Link : [CHATGPT](#)

Type Conversion

In Python, **Type Conversion** means changing one data type into another data type.

Example:

- Integer $\rightarrow$ String
- String $\rightarrow$ Float
- Float $\rightarrow$ Integer

Python has two types of type conversion:

1. Implicit Type Conversion

Python automatically converts one data type into another.

Example:

```
x = 10          # int
y = 2.5         # float

result = x + y

print(result)
print(type(result))
```

Output:

```
12.5
<class 'float'>
```

Here, Python automatically converts 10 (int) into 10.0 (float).

2. Explicit Type Conversion

The programmer manually converts the data type using functions.

Common conversion functions:

Function Converts To

```
int()    Integer
float()  Float
str()    String
```

Function Converts To

`bool()` Boolean

Integer Conversion

```
x = "100"

y = int(x)

print(y)
print(type(y))
```

Output:

```
100
<class 'int'>
```

Float Conversion

```
x = "10.5"

y = float(x)

print(y)
print(type(y))
```

String Conversion

```
x = 25

y = str(x)

print(y)
print(type(y))
```

Boolean Conversion

```
print(bool(1))
print(bool(0))
print(bool(""))
print(bool("Python"))
```

Output:

```
True
False
```

False
True

Important Examples

Float to Integer

```
x = 10.9  
  
y = int(x)  
  
print(y)
```

Output:

10

`int()` removes the decimal part.

User Input Conversion

`input()` always returns data in string format.

```
age = input("Enter age: ")  
  
print(type(age))
```

Output:

<class 'str'>

Convert it into integer:

```
age = int(input("Enter age: "))  
  
print(age)  
print(type(age))
```

Quick Summary

Conversion	Example
String → Integer	<code>int("10")</code>
Integer → String	<code>str(10)</code>
String → Float	<code>float("5.5")</code>
Integer → Float	<code>float(10)</code>

Conversion	Example
Any Value $\rightarrow$ Boolean	<code>bool (value)</code>

Link : [CHATGPT](#)

Data Type

In Python, a **data type** defines the type of value a variable can store.

Example:

```
name = "Krish"  
age = 20  
price = 99.5
```

- "Krish" → String
- 20 → Integer
- 99.5 → Float

Python automatically detects the data type.

You can check the data type using `type()`.

Example:

```
x = 10  
  
print(type(x))
```

Output:

```
<class 'int'>
```

Main Data Types in Python

1. Integer (`int`)

Used for whole numbers.

Example:

```
x = 10  
y = -5  
  
print(type(x))
```

2. Float (`float`)

Used for decimal numbers.

Example:

```
price = 99.5
pi = 3.14

print(type(price))
```

3. String (`str`)

Used for text data.

Strings are written inside quotes.

Example:

```
name = "Krish"
city = 'Rajkot'

print(type(name))
```

4. Boolean (`bool`)

Used for True or False values.

Example:

```
is_student = True
is_logged_in = False

print(type(is_student))
```

Collection Data Types

5. List (`list`)

Used to store multiple items.

- Ordered
- Changeable
- Allows duplicate values

Example:

```
numbers = [10, 20, 30, 40]

print(numbers)
print(type(numbers))
```

6. Tuple (`tuple`)

Similar to list but cannot be changed.

- Ordered
- Unchangeable
- Allows duplicates

Example:

```
data = (1, 2, 3)
print(type(data))
```

7. Set (`set`)

Used to store unique values.

- Unordered
- No duplicate values

Example:

```
nums = {1, 2, 3, 3, 4}
print(nums)
print(type(nums))
```

Output:

```
{1, 2, 3, 4}
```

8. Dictionary (`dict`)

Stores data in `key : value` format.

Example:

```
student = {
    "name": "Krish",
    "age": 20
}

print(student)
print(type(student))
```

None Data Type

9. NoneType (None)

Represents no value.

Example:

```
x = None  
  
print(type(x))
```

Quick Summary Table

Data Type	Example
int	10
float	5.5
str	"Python"
bool	True
list	[1,2,3]
tuple	(1,2,3)
set	{1,2,3}
dict	{"a":1}
NoneType	None

Example Program

```
name = "Krish"  
age = 20  
height = 5.8  
is_student = True  
  
print(type(name))  
print(type(age))  
print(type(height))  
print(type(is_student))
```

Link : CHATGPT

Conditional Statements in Python

Conditional statements allow a program to make decisions based on conditions.

1. `if` Statement

The `if` statement executes a block of code only when the condition is `True`.

Syntax:

```
if condition:
    statement
```

Example:

```
age = 18

if age >= 18:
    print("You are eligible to vote.")
```

Output:

You are eligible to vote.

2. `if-else` Statement

When a condition is `True`, the `if` block runs. Otherwise, the `else` block runs.

Syntax:

```
if condition:
    statement1
else:
    statement2
```

Example:

```
num = 5

if num % 2 == 0:
    print("Even Number")
else:
    print("Odd Number")
```

Output:

Odd Number

3. `if-elif-else` Statement

Used when there are multiple conditions.

Syntax:

```
if condition1:
    statement1
elif condition2:
    statement2
else:
    statement3
```

Example:

```
marks = 75

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
else:
    print("Fail")
```

Output:

Grade B

4. Nested if Statement

An if statement inside another if statement.

Example:

```
age = 20
citizen = True

if age >= 18:
    if citizen:
        print("Eligible to vote")
    else:
        print("Not a citizen")
else:
    print("Underage")
```

Output:

Eligible to vote

Comparison Operators

Used to compare values.

Operator	Meaning	Example
<code>==</code>	Equal to	<code>a == b</code>
<code>!=</code>	Not equal to	<code>a != b</code>
<code>></code>	Greater than	<code>a > b</code>
<code><</code>	Less than	<code>a < b</code>
<code>>=</code>	Greater than or equal to	<code>a >= b</code>
<code><=</code>	Less than or equal to	<code>a <= b</code>

Example:

```
x = 10

if x > 5:
    print("x is greater than 5")
```

Logical Operators

and

Returns `True` if both conditions are true.

```
age = 20

if age >= 18 and age <= 60:
    print("Working age")
```

or

Returns `True` if at least one condition is true.

```
day = "Sunday"

if day == "Saturday" or day == "Sunday":
    print("Weekend")
```

not

Reverses the result.

```
is_raining = False

if not is_raining:
    print("Go outside")
```

Truthy and Falsy Values

Python treats some values as `False` automatically.

Falsy values:

```
False
0
0.0
''
[]
{}
None
```

Example:

```
name = ""

if name:
    print("Name exists")
else:
    print("Name is empty")
```

Output:

```
Name is empty
```

Short-Hand if

```
age = 20

if age >= 18: print("Adult")
```

Ternary Operator (One-line if-else)

Syntax:

```
value_if_true if condition else value_if_false
```

Example:

```
age = 16

result = "Adult" if age >= 18 else "Minor"
print(result)
```

Output:

```
Minor
```

Real-Life Example

```
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "1234":
```

```
        print("Login Successful")
    else:
        print("Invalid Username or Password")
```

Flow of Execution

```
Start
|
Check Condition
|
True -----> Execute IF block
|
False -----> Execute ELSE block
|
End
```

Link : CHATGPT

Loops in Python

Loops are used to execute a block of code repeatedly.

Python has **two main types of loops**:

1. `for` loop
 2. `while` loop
-

1. `for` Loop

A `for` loop is used to iterate over a sequence (list, tuple, string, range, etc.).

Syntax

```
for variable in sequence:
    statements
```

Example 1: Print numbers from 1 to 5

```
for i in range(1, 6):
    print(i)
```

Output:

```
1
2
3
4
5
```

Example 2: Print characters of a string

```
name = "Python"

for ch in name:
    print(ch)
```

Output:

```
P
y
t
h
o
n
```

Example 3: Loop through a list

```
fruits = ["Apple", "Banana", "Mango"]

for fruit in fruits:
    print(fruit)
```

Output:

```
Apple
Banana
Mango
```

`range()` Function

The `range()` function generates a sequence of numbers.

`range(stop)`

```
for i in range(5):
    print(i)
```

Output:

```
0
1
2
3
4
```

`range(start, stop)`

```
for i in range(1, 6):
    print(i)
```

Output:

```
1
2
3
4
5
```

`range(start, stop, step)`

```
for i in range(0, 11, 2):
    print(i)
```

Output:

```
0
2
4
```

6
8
10

2. `while` Loop

A `while` loop executes as long as the condition is `True`.

Syntax

```
while condition:
    statements
```

Example

```
i = 1

while i <= 5:
    print(i)
    i += 1
```

Output:

```
1
2
3
4
5
```

Infinite Loop

If the condition never becomes `False`, the loop runs forever.

```
while True:
    print("Hello")
```

⚠ Press `Ctrl + C` to stop it.

Loop Control Statements

1. `break`

Stops the loop immediately.

```
for i in range(1, 11):
```

```
    if i == 6:
        break
    print(i)
```

Output:

```
1
2
3
4
5
```

2. continue

Skips the current iteration.

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Output:

```
1
2
4
5
```

3. pass

Does nothing. Used as a placeholder.

```
for i in range(5):
    pass
```

Nested Loops

A loop inside another loop.

Example

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i, j)
```

Output:

```
1 1
1 2
```

```
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

Practical Examples

Sum of first 10 numbers

```
total = 0

for i in range(1, 11):
    total += i

print(total)
```

Output:

```
55
```

Factorial using for loop

```
n = 5
fact = 1

for i in range(1, n + 1):
    fact *= i

print(fact)
```

Output:

```
120
```

Fibonacci Series

```
a, b = 0, 1

for i in range(10):
    print(a, end=" ")
    a, b = b, a + b
```

Output:

```
0 1 1 2 3 5 8 13 21 34
```

Difference Between `for` and `while`

`for` Loop

Used when the number of iterations is known

Easier to use with lists, strings, ranges

Less chance of infinite loop

`while` Loop

Used when iterations are unknown

Based on a condition

Can create infinite loops easily

Example

```
# for loop
for i in range(5):
    print(i)

# while loop
i = 0
while i < 5:
    print(i)
    i += 1
```

Link : [CHATGPT](#)

Link : [ALL](#)